PRD: Test Case Runner Website (Library Management System)

Oct 21, 2026 · @Shakthi

1. Overview

The website runs the eight Group 5 test cases against a small built-in Library Management System and records a real Pass or Fail for each, so the results are executed, not assumed.

Problem. The assignment sheet asks for test cases written on paper with "Not Executed" results. A reviewer cannot see whether those cases are precise enough to run, and the group cannot show how the rules (three-book limit, availability, no double return, records) behave.

Goal. Give the group a clean, single-page web app that (1) simulates the library rules, (2) runs each test case in a repeatable way, and (3) exports a results report for the presentation.

Scope. A front-end only application with no server and no login. It holds sample students and books, runs TC-01 to TC-08, and lets a tester also try the actions by hand.

Important rule. The submitted soft copy keeps Actual Result and Status as "Not Executed", as the assignment requires. The website is the supporting demonstration, and its results are shown only after a test is actually run.

Success criteria

Measure	Target
Test cases runnable in the app	8 of 8 (TC-01 to TC-08)
Time to run the full suite	Under 5 seconds
Result per test	Actual Result text and Pass/Fail Status, produced by the run
Same input, same result	100%, because each test resets to a known state
Setup for a new user	Open the page; no install, no sign-in

2. Users and use cases

Three kinds of people use the site, and the main one is the group member who runs and presents the tests.

User	Goal	Key use case
Group member (tester)	Show that the test cases work	Run one test or the whole suite and read the result
Faculty reviewer	Judge the quality of the test cases	See each test's input, expected result, actual result and status side by side
Class audience	Understand the rules	Watch the library state change as a test runs

User stories

1. As a tester, I run all eight tests with one click and see Pass or Fail for each.
2. As a tester, I run a single test and watch its steps, so I can explain it live.
3. As a reviewer, I compare expected and actual results for any test.
4. As a tester, I reset the library to its starting data in one click.
5. As a tester, I try issuing and returning books by hand to see the rules in action.
6. As a group, we export the results as a PDF or CSV for the presentation.

3. Functional requirements

The app has four functional areas: the library simulator, the test runner, the results view and the report export. "Must" items are needed for submission; "Should" items add polish.

ID	Requirement	Priority
FR-01	The simulator holds sample students (S101–S104) and books (B01–B06) with the starting states the test cases need.	Must
FR-02	Issuing a book is allowed only if the book is Available and the student holds fewer than three books.	Must
FR-03	A rejected issue shows a specific message ("book not available" or "borrowing limit of 3 reached") and changes nothing.	Must
FR-04	Returning a book makes it Available and reduces the student's count by one.	Must
FR-05	Returning a book with no active issue is rejected with an "already returned" message and creates no record.	Must
FR-06	Every successful issue and return adds a transaction record with student, book, action and date; rejected attempts add none.	Must
FR-07	The runner lists TC-01 to TC-08 with ID, objective, input/action and expected result, matching the soft copy.	Must
FR-08	Before each test the runner resets the library, applies the test's precondition, performs the action, and compares the outcome with the expected result.	Must
FR-09	The runner shows an Actual Result sentence and a Status (Not Executed, Passed or Failed) for each test, and each starts as Not Executed.	Must
FR-10	"Run all" executes the suite in order, and "Run" executes a single test.	Must

ID	Requirement	Priority
FR-11	A summary shows total, passed, failed and not-executed counts.	Must
FR-12	A manual panel lets the tester issue and return books and view the live book, student and transaction tables.	Should
FR-13	The report exports as CSV and as a print-ready page (save as PDF).	Should
FR-14	Results persist in the browser between visits, and "Reset all" clears them.	Should
FR-15	Each test can be expanded to show its step-by-step trace (setup, action, checks).	Should

4. Non-functional requirements and design principles

The site should feel calm and professional: simple layout, readable type, and no surprises.

ID	Area	Requirement
NFR-01	Performance	Full suite runs in under 5 seconds; first page load under 2 seconds on a normal connection.
NFR-02	Reliability	Tests are deterministic: each starts from a reset state and never depends on a previous test's leftovers.
NFR-03	Usability	A new user can run all tests within 10 seconds of opening the page, with no instructions.
NFR-04	Accessibility	Keyboard operable, visible focus, status shown with text and icon, not colour alone, and contrast of at least 4.5:1.
NFR-05	Responsiveness	Works from a 360 px phone to a projector screen.
NFR-06	Compatibility	Current Chrome, Edge, Firefox and Safari; works offline once loaded.

ID	Area	Requirement
NFR-07	Maintainability	Rules, tests and interface live in separate modules; adding a test case means adding one data entry.
NFR-08	Privacy	All data is sample data kept in the browser; nothing is sent anywhere.

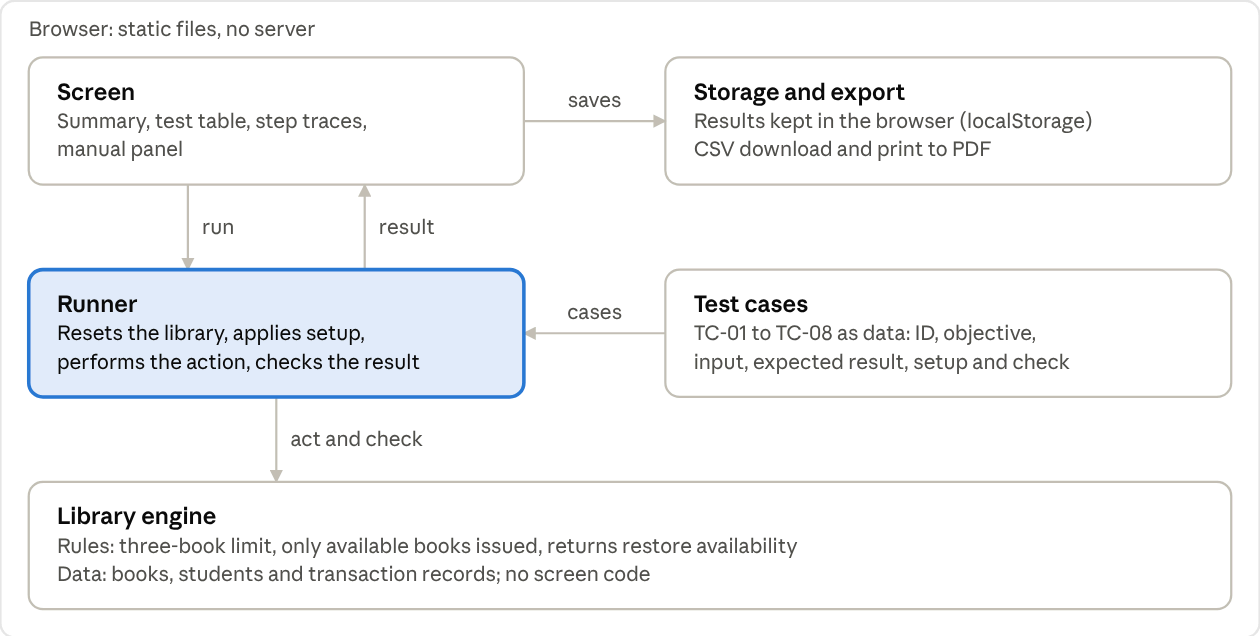
Design principles

- **Clarity first.** One screen: summary on top, test table below, details on demand.
- **Calm visual style.** White background, one accent colour, green and red used only for Passed and Failed, a system font stack, and an 8 px spacing grid.
- **Truthful status.** A test is Not Executed until it is run; the interface never shows a result that was not produced by a run.
- **Show the reason.** Failed tests show expected versus actual, so the cause is visible at once.
- **Small and boring code.** Plain HTML, CSS and JavaScript with no build step, so it opens anywhere and is easy to explain to the faculty.

5. Architecture and tech stack

The site is a set of static files that run entirely in the browser, split into four modules so rules, tests and screen can change independently.

Rules, tests and screen live in separate modules



architecture · 4 modules, 1 data store

The Screen asks the Runner to run a test. The Runner loads the case from Test cases, acts on the Library engine, and returns a result the Screen displays and saves. The manual panel calls the Library engine directly.

Tech stack

Layer	Choice	Why
Markup and style	HTML5 and CSS with custom properties	No build step; colours and spacing are defined once as tokens
Logic	Plain JavaScript, classic scripts (no modules)	Opens from a local file and works offline
Storage	Browser localStorage	Keeps results between visits with no server
Export	CSV from a Blob, print stylesheet for PDF	No libraries needed
Fonts	System font stack	Fast, consistent and no downloads

Layer	Choice	Why
Hosting	GitHub Pages or Netlify, plus a zipped backup	Free, one link to submit

6. Build process, step by step

The build takes ten steps and roughly six hours (planning estimates, not measured). Each step has a clear output, so the team can split the work and check progress.

```
library-test-runner/
  index.html        page structure
  styles.css        design tokens and layout
  js/library.js     library rules (no screen code)
  js/tests.js       the 8 test cases as data
  js/runner.js      runs tests and compares results
  js/app.js         screen, storage and export
```

1. **Set up the project (15 min).** Create the folder above and open `index.html` in a browser. *Why:* a fixed structure keeps rules, tests and screen separate, so each can be explained and changed alone.
2. **Build the library engine (45 min).** In `library.js`, write `createLibrary()`, `issueBook()` and `returnBook()`. Each returns a result with `ok` and a `message`, and never touches the screen. *Why:* rules without screen code are easy to test and easy to present.
3. **Write the test cases as data (30 min).** In `tests.js`, give each of TC-01 to TC-08 an ID, objective, input text, expected text, a setup, an action and a check. *Why:* adding or changing a test then means editing one entry, and the text matches the soft copy exactly.
4. **Write the runner (45 min).** In `runner.js`, for each test: reset the library, apply the setup, perform the action, run the check, and return the Actual Result sentence and Passed or Failed. *Why:* resetting first makes every result repeatable.
5. **Build the screen (90 min).** Add the summary cards, the test table, the Run and Run all buttons, expandable step traces and the manual panel. *Why:* this is what the audience sees, so it follows the design principles in section 4.

6. **Add storage and export (30 min).** Save results in `localStorage`; add CSV download and a print stylesheet for PDF. *Why:* results survive a page refresh and the group gets a file for submission.
7. **Polish and accessibility (45 min).** Check keyboard use, focus outlines, contrast, text labels on statuses and the 360 px layout. *Why:* a clean, smooth finish is the difference between a student project and senior-level work.
8. **Verify the tests (30 min).** Run all eight; all must pass. Then break a rule on purpose (change the limit check from "fewer than 3" to "3 or fewer") and confirm TC-04 fails. Restore the rule. *Why:* a test that cannot fail proves nothing, and this demonstrates the defect described in Part C.
9. **Deploy (15 min).** Publish the folder on GitHub Pages or Netlify and open the link on a phone. Keep the zipped folder as a backup that opens offline. *Why:* a link is easy to submit, and the backup covers a classroom with no internet.
10. **Rehearse (30 min).** Each member practises their part: run one test live and explain expected versus actual. *Why:* every member must contribute and be ready to present.

7. Acceptance criteria, risks and scope

The site is done when every Must requirement works and the checklist below passes. Each test builds its own sample data in its setup step (for example, TC-04 gives S103 three issued books), so no test depends on another.

Acceptance criteria

- [ ] All eight tests appear with the same ID, objective, input and expected result as the soft copy
- [ ] Every test shows Not Executed until it is run
- [ ] Run all gives eight Passed on the correct rules
- [ ] With the limit rule deliberately broken, TC-04 shows Failed with expected versus actual
- [ ] Rejected actions create no transaction record (TC-02, TC-04, TC-06)
- [ ] Reset restores the starting data and clears results
- [ ] CSV export and print-to-PDF both contain all eight rows
- [ ] Layout works at 360 px and on a projector, and the page loads with no console errors

Risks

Risk	Impact	Mitigation
Time runs short before submission	Website incomplete	Build Must items first (steps 1–4, 5, 8); skip Should items if needed
No internet in class	Hosted link unreachable	Keep a zipped copy that opens offline
Website results contradict the soft copy	Confusion in marking	State that the soft copy is the paper-based record and the site is a demonstration
A member cannot explain the code	Marks lost for contribution	Assign steps by member and rehearse in step 10

Out of scope

- Real user accounts, login or a database
- Fines, due dates, reservations, search and book catalogue management
- Testing any real library system or sending data to a server